
어떤 상품을 캐시해야 할까

Sliding Window 기반 선택적 캐시 설계

Tech Talk 15

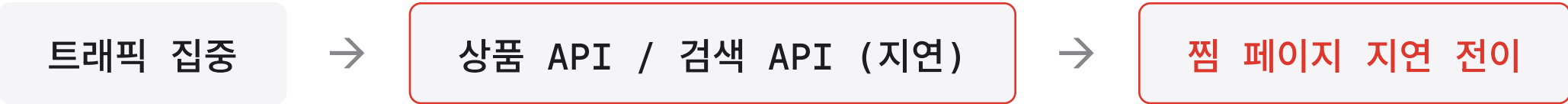
SIPE 5th 이상원

목차

- 01 문제 상황
- 02 리스트 캐시와 전체 상품 캐시의 한계
- 03 선택적 캐시라는 방향
- 04 고정 윈도우의 한계
- 05 Sliding Window 기반 설계
- 06 운영 가설과 모니터링 지표
- 07 왜 적용하지 않았는가
- 08 결론

트래픽 집중으로 상품 API, 검색 API 지연이 찜 페이지로 전이됐다

- 트래픽이 몰리면서 상품 API와 검색 API에서 지연이 발생했다
- 찜 페이지는 상품 정보를 상품 API, 검색 API 호출로 조합했다
- 하위 API 지연이 찜 페이지 응답 지연으로 직접 전이됐다
- 장애 전파를 줄이기 위한 대응이 필요했다



리스트 자체를 캐시 한다면?

- 찜 페이지 결과를 리스트 단위로 캐시
- 하지만 사용자별 조합이 모두 다름
- 페이지 단위 캐시는 재사용성이 낮음
- 리스트 캐시는 효율이 크지 않음



낮은 재사용성

모든 상품 정보 캐시는 적절한 해법이 아니었다

- 서로 다른 리스트도 결국 동일한 상품 정보를 참조한다
- 초기 해법은 상품 단위 캐시 적용이었음
- 전체 상품 캐시는 효율 대비 비용이 과도하다
- 캐시의 핵심 지표는 저장량이 아니라 hit ratio다

전체 상품 캐시는 hit ratio 관점에서 비효율적

800만

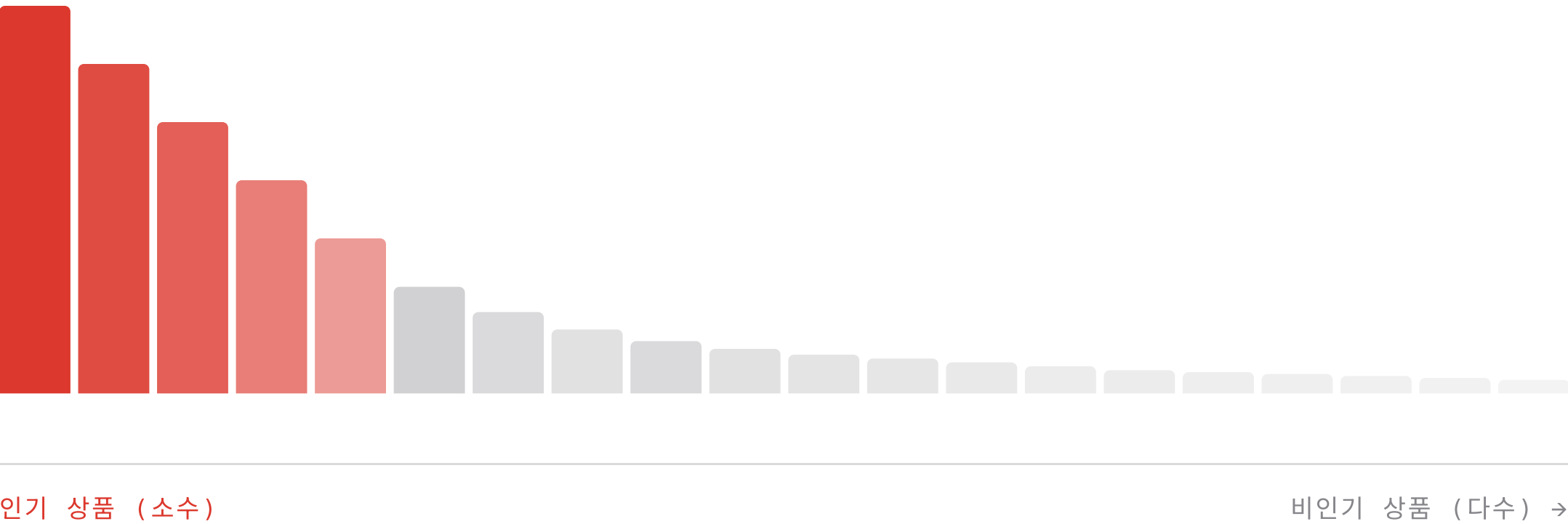
찜 데이터 row 수

~200개

10회 이상 중복 찜된 상품 수

80%+

낮은 중복도 비중



캐시의 목적은 저장이 아니라 반복 조회 데이터를 빠르게 재사용하는 데 있다

Long Tail 구조 확인

상품 조회 분포는 소수의 인기 상품과 다수의 비인기 상품으로 구성된 롱테일 구조에 가까웠다.

따라서 전체 상품 정보를 캐시하기보다 반복 조회되는 일부 상품만 선별적으로 캐시하는 편이 더 효율적이었다.

선택적 캐시 → hit ratio 극대화

캐시 대상은 반복 조회 상품으로 좁혀야 했다

- 전체 캐시보다 선택적 캐시가 적합했다
- 반복적으로 조회되는 상품만 캐시 대상으로 삼았다
- ➔ 캐시 가치 판단 기준 수립이 핵심 과제였다
- 인기 상품을 식별할 명확한 기준이 필요했다

고정 윈도우 방식은 자주 조회되는 상품의 연속성을 반영하지 못했다

- 일정 시간 단위로 카운터를 집계했다
- 구현은 단순했다
- 하지만 TTL 만료 시 카운터가 한 번에 초기화됐다
- 이 방식은 자주 조회되는 상품의 연속성을 반영하지 못했다

12:59까지 집중 조회된 상품도 1:00이 되면 즉시 비인기 상품처럼 보이는 한계가 있었다

Sliding Window 기반 선택적 캐시 설계

1

상품 조회가 발생

2

해당 상품 버킷 카운트를 증가

3

최근 구간 카운트를 합산

4

임계치 초과 시 상품 정보를 캐시

5

찜 페이지는 캐시 hit 시 즉시 응답

조회 버킷 집계 구조를 설계



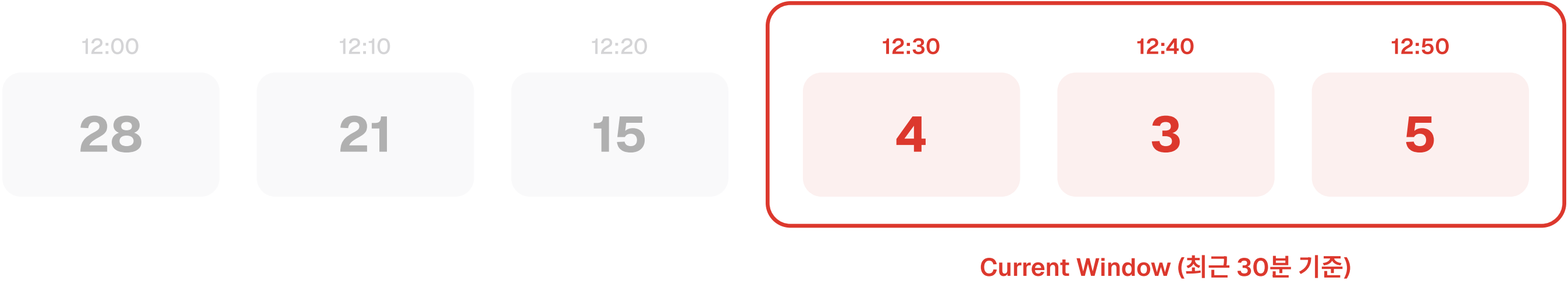
상품 찜 조회가
버킷 집계 시작점



조회 버킷 생성 및 갱신	
Product A	count = 28
Product B	count = 21
Product C	count = 15

10분 단위 버킷

Sliding Window 합산 방식을 적용



합산 결과: $4 + 3 + 5 = 12$

임계치(10) 초과 → 캐시 대상

선택적 캐시 적용 흐름을 구성

조회 발생 → 버킷 집계 → 임계치 판별 → 캐시 반영 → 후속 조회 가속

1

찜 페이지
조회 발생

사용자 요청이
집계 흐름을 시작함



2

조회 버킷
카운트 누적

상품별 시간 버킷에
조회 수를 반영



3

Sliding Window
구간 합산

최근 N개 버킷 기준으로
인기도를 계산



4

임계치 판별
및 캐시 반영

합산값이 기준 이상이면
비동기로 캐시 저장



5

후속 조회에서
캐시 hit 유도

다음 요청부터는
캐시 응답 경로 사용

Cache Hit 경로

찜 페이지 요청



Redis 캐시 조회



캐시 Hit! 즉시 반환

Cache Miss 경로 (비동기 캐시 저장)

찜 페이지 요청



버킷 카운트 +1



구간 합산



임계치 초과?



비동기 캐시 저장

이 설계는 운영 가설 검증을 위한 대안

- 목표는 즉시 적용이 아니라 효과 검증이었음
- 효과가 확인되면 유지하고 아니면 스위치로 비활성화하는 전제였음
- 캐시 도입 자체보다 도입 기준을 세우는 일이 더 중요했음

 Switch/Toggle 기반으로 부담 없이 켜고 끌 수 있는 운영 구조를 전제로 함

적용을 위한 모니터링 지표

01 캐시 효율성

- cache hit ratio 추이
- miss ratio 변화
- 임계치 초과 상품 수
- 캐시 대상 상품 비중

02 트래픽 흡수 정도

- 상품 상세 API 호출 감소량
- 검색 API 호출 감소량
- 페이지 진입당 하위 API 호출 수 변화

03 응답 성능

- 응답 시간 p50 / p95 / p99
- 타임아웃 비율
- 에러율 변화
- Redis read/write latency

04 리소스 및 비용

- Redis memory usage
- bucket key 증가 추이
- eviction 발생 여부

설계는 가능했지만 낮은 적용 필요성

- 리뉴얼 이후 상품 상세 API가 먼저 개선됨
- 관련 모듈 API도 함께 최적화됨
- 예상했던 병목이 운영 환경에서 크게 드러나지 않았음
- 운영 복잡도를 감수할 만큼의 우선순위는 아니었음

| 캐시 자체보다 중요했던 것은, 무엇을 언제 캐시할지 판단하는 기준이었음

Thank you